¡Ya casi empezamos!



Centro de e-Learning

UTN.BA

Clase 6

Hooks intermedios y Composición Lógica

Centro de e-Learning
UTN.BA

¿Dudas con el Contenido previo?

Centro de e-Learning
UTN.BA

Objetivos de la Clase

- **Aprender** a usar useRef, useMemo, useCallback, de forma cómoda y constante con React Native.
- **Crear** custom hooks y separar la lógica de la UI en proyectos de RN.

¿Qué son los Hooks Intermedios?

Centro de e-Learning
UTN.BA

¿Qué son los Hooks Intermedios?

Son herramientas que permiten optimizar funciones, evitar renders innecesarios, almacenar valores entre renders y reutilizar lógica.

Repasemos algunos
Hooks

Centro de
e-Learning
UTN.BA

¿Qué es useEffect?

useEffect es un hook que te permite ejecutar efectos secundarios en componentes funcionales: llamadas a APIs, suscripciones, timers, listeners, sincronizar estado con el exterior, etc.

¿Cómo funciona?

efecto: función que se ejecuta después de renderizar.

dependencias (array opcional): le dice a React cuándo volver a ejecutar el efecto.

Si efecto devuelve una función, esa es la limpieza (cleanup).

```
useEffect(efecto, dependencias?)
```

Cuándo se ejecuta:

Sin array: en cada render (rara vez útil).

Array vacío []: una vez tras el primer render (montaje).

Con deps [a, b]: en el primer render y cada vez que cambia a o b.

Limpieza: antes de desmontar y antes de re-ejecutar el efecto si cambió una dependencia.

Reglas importantes

Decláralo siempre en el tope del componente (no dentro de if/loops).

Incluye todas las variables usadas dentro del efecto en el array de dependencias (o memorízalas con useCallback/useMemo).

Si el efecto crea algo externo (timer, listener, suscripción), devuelve una limpieza que lo desregistre.

```
useEffect(() => {
  const ctrl = new AbortController();
  const t = setTimeout(() => ctrl.abort(), 10000);

  (async () => {
    try {
      const r = await fetch('https://api.example.com/items', { signal: ctrl.signal });
      if (!r.ok) throw new Error('HTTP ' + r.status);
      // setState(await r.json());
    } catch (e) {
      // manejar error o abort
    }
  })();

  return () => { clearTimeout(t); ctrl.abort(); };
}, []); // una vez
```

```
useEffect(() => {
  // Recalcula o sincroniza cuando cambia "query"
}, [query]);
```

¿Qué es useRef?

Permite guardar valores mutables entre renders sin causar re-render. Se usa para referencias visuales, animaciones o timers.

```jsx
import { useRef } from 'react';
import { TextInput, Button } from 'react-native';

export default function Formulario() {
  const inputRef = useRef(null);

  const enfocarInput = () => {
    inputRef.current?.focus();
  };

  return (
    <>
      <TextInput ref={inputRef} placeholder="Tu nombre" />
      <Button title="Enfocar" onPress={enfocarInput} />
    </>
  );
}
```

¿Qué es useMemo?

Memoriza el resultado de funciones costosas, evitando recálculos innecesarios si las dependencias no cambian.

```
const usuariosFiltrados = useMemo(() => {
  return listaUsuarios.filter(u => u.edad > 18);
}, [listaUsuarios]);
```

¿Qué es useCallback?

Memoriza funciones para evitar que se vuelvan a crear en cada render. Útil cuando se pasan funciones a componentes hijos.

```
const incrementar = useCallback(() => {
  setContador(prev => prev + 1);
}, []);
```

Diferentes entre useMemo y useCallback

Hook	Devuelve	Uso principal
useMemo	Resultado	Memorizar cálculos
useCallback	Función	Memorizar funciones

¡Es hora de un descansito!

Volvemos en *10'* a seguir *Practicando*



Centro de e-Learning

UTN.BA

Custom
Hooks

Centro de
e-Learning
UTN.BA

¿Qué es un Custom Hook?

Es una función reutilizable que comienza con use y encapsula lógica con estado o efectos. Mejora la legibilidad y reusabilidad del código.

```jsx
import { useState } from 'react';

export function useContador(inicial = 0) {
  const [valor, setValor] = useState(inicial);

  const incrementar = () => setValor(prev => prev + 1);
  const decrementar = () => setValor(prev => prev - 1);
  const resetear = () => setValor(inicial);

  return { valor, incrementar, decrementar, resetear };
}
```

```jsx
const { valor, incrementar } = useContador(5);
```

Beneficios de usar custom hooks

- Reutilización de lógica compleja
- Código más limpio y mantenible
- Mejor separación de responsabilidades
- Permiten testear lógica sin renderizar UI

```
import { useEffect, useState } from 'react';

export function useCargarDatos(url) {
  const [datos, setDatos] = useState([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch(url)
      .then(resp => resp.json())
      .then(setDatos)
      .catch(setError)
      .finally(() => setCargando(false));
  }, [url]);

  return { datos, cargando, error };
}
```

¿Qué es la composición lógica?

Separar la lógica del componente visual, organizando funcionalidades en hooks reutilizables. Evita componentes gigantes con demasiada responsabilidad.

Buenas prácticas con hooks

- Todos los custom hooks deben comenzar con use
- Nunca uses hooks dentro de condiciones o bucles
- Extraé y documentá hooks reutilizables
- Usá useCallback y useMemo en listas o componentes hijos frecuentes.

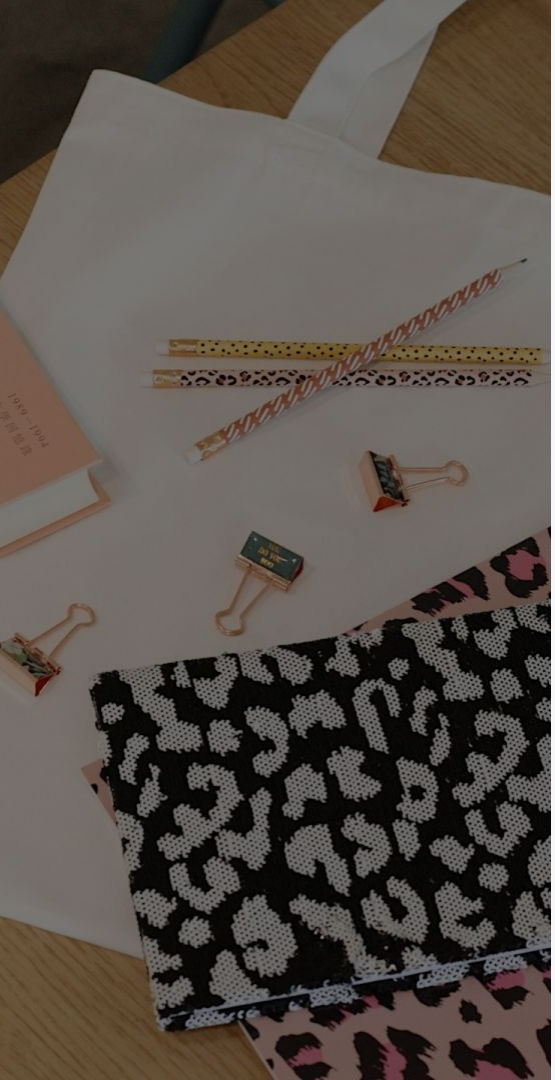
¿Preguntas?
Debatamos un poco los temas de hoy.



Centro de e-Learning
UTN.BA

¡El resto de la clase será práctica!

Centro de e-Learning
UTN.BA



Clase 6

Actividad de Hoy

Centro de e-Learning

UTN.BA

Actividad de Hoy

Formato

Esta actividad no es de entrega obligatoria, y puede realizarse únicamente con código sin necesidad de una presentación o archivo de texto.

Consigna

Crear un custom hook useFetch para consumir una API con GET, mostrar estado de carga y errores.

¡Nos vemos la próxima clase!

“Theming y Gestión Global de Estilos”

Centro de e-Learning

UTN.BA

Bibliografía

- https://react.dev/reference/react
- https://reactnative.dev/docs/getting-started
- https://kentcdodds.com/blog/usememo-and-usecallback